BIT 326 - Distributed Systems

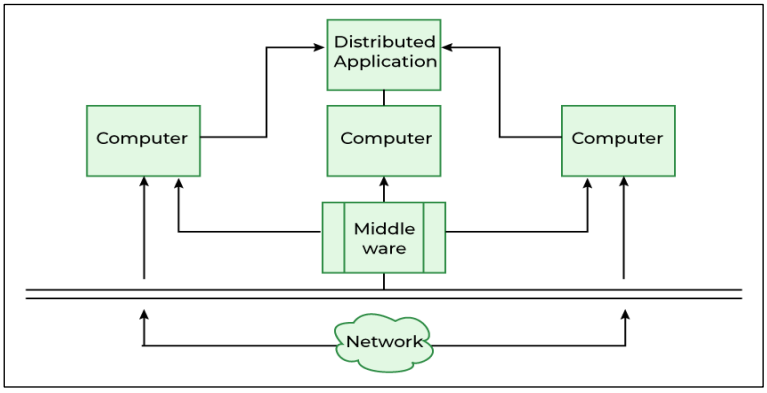
Chapter 2 – Distributed Systems Architecture

Distributed systems are built up on top of existing networking and operating systems software. A distributed system architecture is a design where multiple independent computers (nodes) work as a single, unified system, sharing resources and coordinating tasks over a network to achieve common goals, offering key benefits like scalability, fault tolerance, and high performance by spreading workloads instead of relying on one server.

2.1. Categories

Distributed systems architecture can be categorised as follows;

1) By Communication & Structure
   a) Client-Server: A fundamental model where clients request resources/services from a central server, which processes the request and sends back a response (e.g., web browsing).
   b) Peer-to-Peer (P2P): A decentralized model where each node acts as both client and server, sharing resources directly with other nodes (e.g., BitTorrent, blockchain).
   c) Three-Tier/N-Tier: Separates applications into presentation, logic (middle tier), and data layers for scalability (e.g., typical web apps).
   d) Microservices: An application built as a collection of small, independent, loosely coupled services that communicate via APIs.
   e) Service-Oriented Architecture (SOA): Similar to microservices but often involves larger, broader services and an Enterprise Service Bus (ESB).
   f) Event-Driven: Components communicate by producing and reacting to events, enabling asynchronous interactions.
2) By Purpose & Deployment
   a) Distributed Computing Systems: Link multiple networked computers to work as one powerful system, sharing data and processing power for complex tasks.



Focus on computation, including:

- Cluster Computing: High-performance computing using a network of similar machines. connects multiple similar computers (nodes) via a high-speed network to work together as a single, powerful system (e.g., supercomputers).
- Grid Computing: Connects geographically dispersed computers to solve large problems.
- Cloud Computing: On-demand access to shared computing resources over the internet.

   b) Distributed Information Systems: consist of multiple independent computers working together as a unified system Focus on data sharing and processing (e.g., distributed databases, file systems like HDFS).

c) Distributed Pervasive Systems: Blends ubiquitous computing with distributed systems, creating interconnected networks of small, often mobile, battery-powered devices (sensors, appliances, trackers) that seamlessly gather data and provide context-aware services without constant user input, aiming for self-configuration, management, and personalized experiences in smart homes, cities, and healthcare. Example Real-time systems from IoT devices, sensors, etc..

3) Other Architectures
   - **Middleware-Based:** Middleware acts as a bridge between different applications/OSs.
   - **Shared-Nothing:** Each node operates independently without sharing resources like memory or storage.

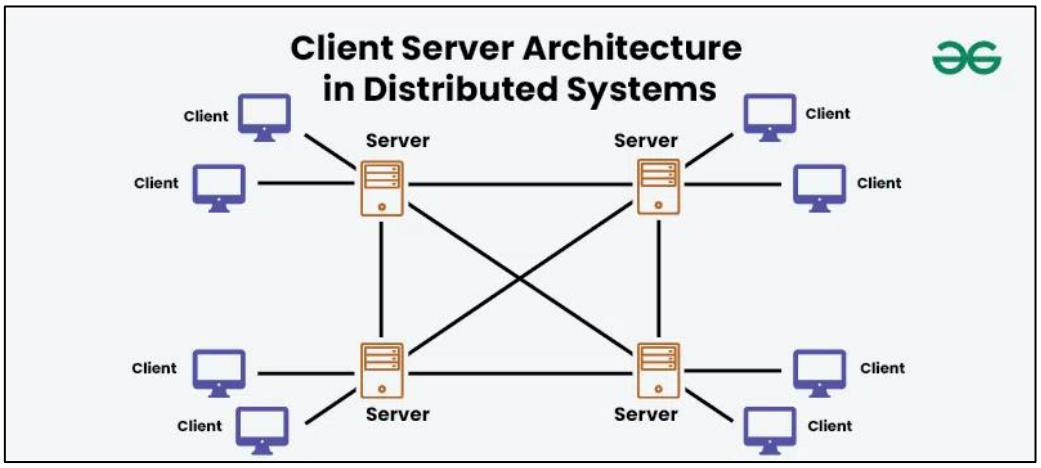
2.2. Architecture Styles in Distributed Systems

Distributed system architecture styles organize components into logical structures like Layered, Client-Server, Peer-Peer, Data-Centered, Event-Driven, and Microservices, each defining how components interact (e.g., hierarchy, decentralization, data focus, event response, independent services) for different goals like modularity, scalability, or flexibility.

2.2.1. Client Server Architecture in Distributed Systems

Client-Server Architecture is a foundational model in distributed systems where the system is divided into two main components: clients and servers. This architecture defines how tasks and services are distributed across different entities within a network.

1) In Client-Server Architecture, clients request services or resources, while servers provide those services or resources.
2) The client initiates a request to the server, which processes the request and returns the appropriate response.
3) This model centralizes the management of resources and services on the server side, while the client side focuses on presenting information and interacting with users.



Client Server Architecture

Key Principles of Client Server Architecture in Distributed Systems

1) **Separation of Concerns**:
   a) **Function**: Clients handle user interactions and requests, while servers manage resources, data, and business logic.

   b) **Principle**: Separates user interface and client-side processing from server-side data management and processing, leading to a clear division of responsibilities.
2) **Centralized Management**:
   a) **Function**: Servers centralize resources and services, making them accessible to multiple clients.
   b) **Principle**: Simplifies resource management and maintenance by concentrating them in one or more server locations.
3) **Request-Response Model**:
   a) **Function**: Clients send requests to servers, which process these requests and send back responses.
   b) **Principle**: Defines a communication pattern where the client and server interact through a well-defined protocol, often using HTTP or similar standards.
4) **Scalability**:
   a) **Function**: Servers can be scaled to handle increasing numbers of clients or requests.
   b) **Principle**: Servers can be upgraded or expanded to improve performance and accommodate growing demand.
5) **Security**:
   a) **Function**: Security mechanisms are often implemented on the server side to control access and manage sensitive data.
   b) **Principle**: Centralizes security policies and controls, making it easier to enforce and manage security measures.

Advantages and Disadvantages of Client Server Architecture in Distributed Systems

Advantages:

1) **Centralized Control**: Easier to manage and update resources and services from a central location.
2) **Simplified Maintenance**: Updates and changes are made on the server side, reducing the need for client-side modifications.
3) **Resource Optimization**: Servers can be optimized for performance and reliability, serving multiple clients efficiently.
4) **Security Management**: Centralized security policies and controls make it simpler to protect resources and data.

Disadvantages:

1) **Single Point of Failure**: Servers can become a single point of failure, impacting all connected clients if they go down.
2) **Scalability Challenges**: Handling a large number of client requests can overwhelm servers, requiring careful load management and scaling strategies.
3) **Network Dependency**: Clients depend on network connectivity to access server resources, which can impact performance and reliability.
4) **Performance Bottlenecks**: High demand on servers can lead to performance bottlenecks, requiring efficient resource management and optimization.

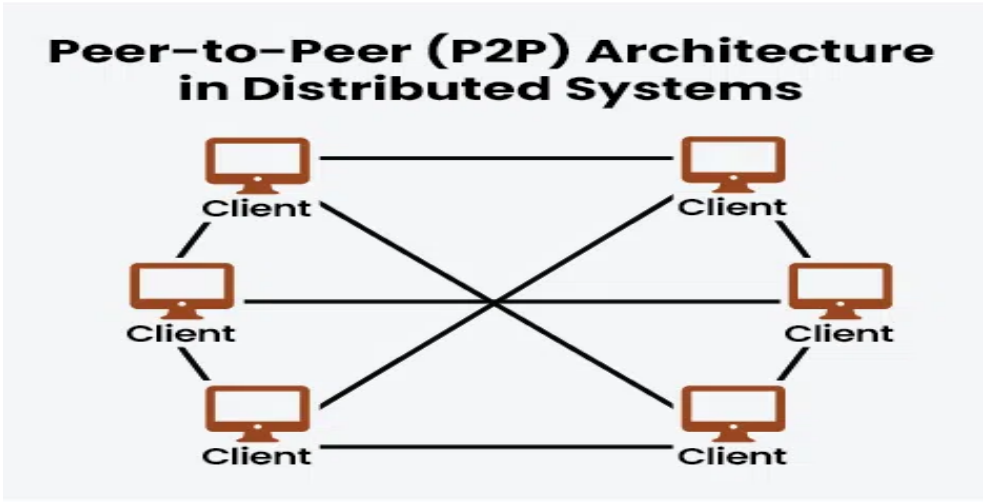
Examples of Client Server Architecture in Distributed Systems

1) **Web Applications**: In a typical web application, web browsers (clients) request web pages or data from web servers.
2) **Email Systems**: Email clients connect to email servers to send, receive, and manage email messages.
3) **Database Access**: Database clients request data and perform queries on database servers.

2.2.2. Peer-to-Peer (P2P) Architecture in Distributed Systems

Peer-to-Peer (P2P) Architecture is a decentralized network design where each node, or "peer," acts as both a client and a server, contributing resources and services to the network. In a P2P architecture, all nodes (peers) are equal participants in the network, each capable of initiating and receiving requests.

- Peers collaborate to share resources, such as files or computational power, without relying on a central server.



Peer-to-Peer (P2P) Architecture

Key Features of Peer-to-Peer (P2P) Architecture in Distributed Systems

1) Decentralization
   - Function: There is no central server or authority. Each peer operates independently and communicates directly with other peers.
   - Advantages: Reduces single points of failure and avoids central bottlenecks, enhancing robustness and fault tolerance.
2) Resource Sharing
   - Function: Peers share resources such as processing power, storage space, or data with other peers.
   - Advantages: Increases resource availability and utilization across the network.
3) Scalability
   - Function: The network can scale easily by adding more peers. Each new peer contributes additional resources and capacity.
   - Advantages: The system can handle growth in demand without requiring significant changes to the underlying infrastructure.
4) Self-Organization

- Function: Peers organize themselves and manage network connections dynamically, adapting to changes such as peer arrivals and departures.
- Advantages: Facilitates network management and resilience without central coordination.

Advantages of Peer-to-Peer (P2P) Architecture in Distributed Systems

1) Fault Tolerance: The decentralized nature ensures that the failure of one or several peers does not bring down the entire network.
2) Cost Efficiency: Eliminates the need for expensive central servers and infrastructure by leveraging existing resources of the peers.
3) Scalability: Easily accommodates a growing number of peers, as each new peer enhances the network's capacity.

Disadvantages of Peer-to-Peer (P2P) Architecture in Distributed Systems

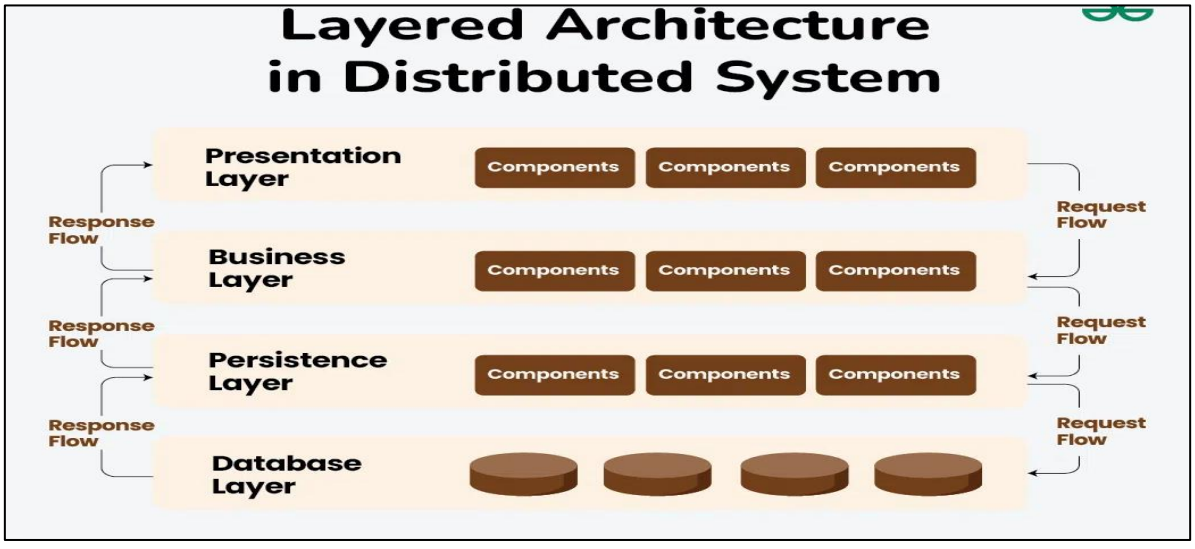
1) Security: Decentralization can make it challenging to enforce security policies and manage malicious activity, as there is no central authority to oversee or control the network.
2) Performance Variability: The quality of services can vary depending on the peers' resources and their availability, leading to inconsistent performance.
3) Complexity: Managing connections, data consistency, and network coordination without central control can be complex and may require sophisticated protocols.

Examples of Peer-to-Peer (P2P) Architecture in Distributed Systems

1) File Sharing Networks: Systems like BitTorrent allow users to share and download files from multiple peers, with each peer contributing to the upload and download processes.
2) Decentralized Applications (DApps): Applications that run on decentralized networks, leveraging P2P architecture for tasks like data storage and computation.

2.2.3. Layered Architecture in Distributed Systems

Layered Architecture in distributed systems organizes the system into hierarchical layers, each with specific functions and responsibilities. This design pattern helps manage complexity and promotes separation of concerns.



Layered Architecture in Distributed Systems

Here's a detailed explanation:

- In a layered architecture, the system is divided into distinct layers, where each layer provides specific services and interacts only with adjacent layers.
- This separation helps in managing and scaling the system more effectively.

Layers and Their Functions

1) **Presentation Layer**
   - **Function**: Handles user interaction and presentation of data. It is responsible for user interfaces and client-side interactions.
   - **Responsibilities**: Rendering data, accepting user inputs, and sending requests to the underlying layers.
2) **Application Layer -Business Layer**
   - **Function**: Contains the business logic and application-specific functionalities.
   - **Responsibilities**: Processes requests from the presentation layer, executes business rules, and provides responses back to the presentation layer.
3) **Middleware Layer – Persistence Layer**
   - **Function**: Facilitates communication and data exchange between different components or services.
   - **Responsibilities**: Manages message passing, coordination, and integration of various distributed components.
4) **Data Access Layer**
   - **Function**: Manages data storage and retrieval from databases or other data sources.
   - **Responsibilities**: Interacts with databases or file systems, performs data queries, and ensures data integrity and consistency.

Advantages of Layered Architecture in Distributed System

1) **Separation of Concerns**: Each layer focuses on a specific aspect of the system, making it easier to develop, test, and maintain.
2) **Modularity**: Changes in one layer do not necessarily affect others, allowing for more flexible updates and enhancements.
3) **Reusability**: Layers can be reused across different applications or services within the same system.
4) **Scalability**: Different layers can be scaled independently to handle increased load or performance requirements.

Disadvantages of Layered Architecture in Distributed System

1) **Performance Overhead**: Each layer introduces additional overhead due to data passing and processing between layers.
2) **Complexity**: Managing interactions between layers and ensuring proper integration can be complex, particularly in large-scale systems.
3) **Rigidity**: The strict separation of concerns might lead to rigidity, where changes in the system's requirements could require substantial modifications across multiple layers.

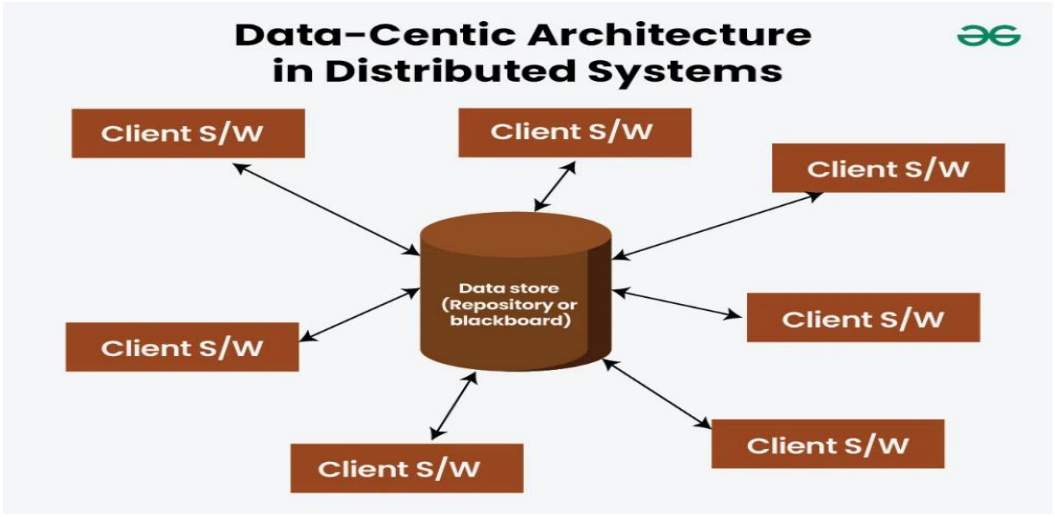
Examples of Layered Architecture in Distributed System

1) **Web Applications**: A common example includes web applications with a presentation layer (user interface), application layer (business logic), and data access layer (database interactions).
2) **Enterprise Systems**: Large enterprise systems often use layered architecture to separate user interfaces, business logic, and data management.

2.2.4. Data-Centic Architecture in Distributed Systems

Data-Centric Architecture is an architectural style that focuses on the central management and utilization of data. In this approach, data is treated as a critical asset, and the system is designed around data management, storage, and retrieval processes rather than just the application logic or user interfaces.

- The core idea of Data-Centric Architecture is to design systems where data is the primary concern, and various components or services are organized to support efficient data management and manipulation.
- Data is centrally managed and accessed by multiple applications or services, ensuring consistency and coherence across the system.



Data-Centic Architecture

Key Principles of Data-Centic Architecture in Distributed Systems

1) **Centralized Data Management**:
   - **Function**: Data is managed and stored in a central repository or database, making it accessible to various applications and services.
   - **Principle**: Ensures data consistency and integrity by maintaining a single source of truth.
2) **Data Abstraction**:
   - **Function**: Abstracts the data from the application logic, allowing different services or applications to interact with data through well-defined interfaces.
   - **Principle**: Simplifies data access and manipulation while hiding the underlying complexity.
3) **Data Normalization**:
   - **Function**: Organizes data in a structured manner, often using normalization techniques to reduce redundancy and improve data integrity.
   - **Principle**: Enhances data quality and reduces data anomalies by ensuring consistent data storage.
4) **Data Integration**:

- **Function**: Integrates data from various sources and systems to provide a unified view and enable comprehensive data analysis.
- **Principle**: Supports interoperability and facilitates comprehensive data analysis across diverse data sources.

5) **Scalability and Performance**:
   - **Function**: Designs the data storage and management systems to handle increasing volumes of data efficiently.
   - **Principle**: Ensures the system can scale to accommodate growing data needs while maintaining performance.

Advantages and Disadvantages of Data-Centic Architecture in Distributed Systems

Advantages:

1) Consistency: Centralized data management helps maintain a single source of truth, ensuring data consistency across the system.
2) **Integration**: Facilitates easy integration of data from various sources, providing a unified view and enabling better decision-making.
3) **Data Quality**: Data normalization and abstraction help improve data quality and reduce redundancy, leading to more accurate and reliable information.
4) **Efficiency**: Centralized management can optimize data access and retrieval processes, improving overall system efficiency.

Disadvantages:

1) **Single Point of Failure**: Centralized data repositories can become a bottleneck or single point of failure, potentially impacting system reliability.
2) **Performance Overhead**: Managing large volumes of centralized data can introduce performance overhead, requiring robust infrastructure and optimization strategies.
3) **Complexity**: Designing and managing a centralized data system can be complex, especially when dealing with large and diverse datasets.
4) **Scalability Challenges**: Scaling centralized data systems to accommodate increasing data volumes and access demands can be challenging and may require significant infrastructure investment.

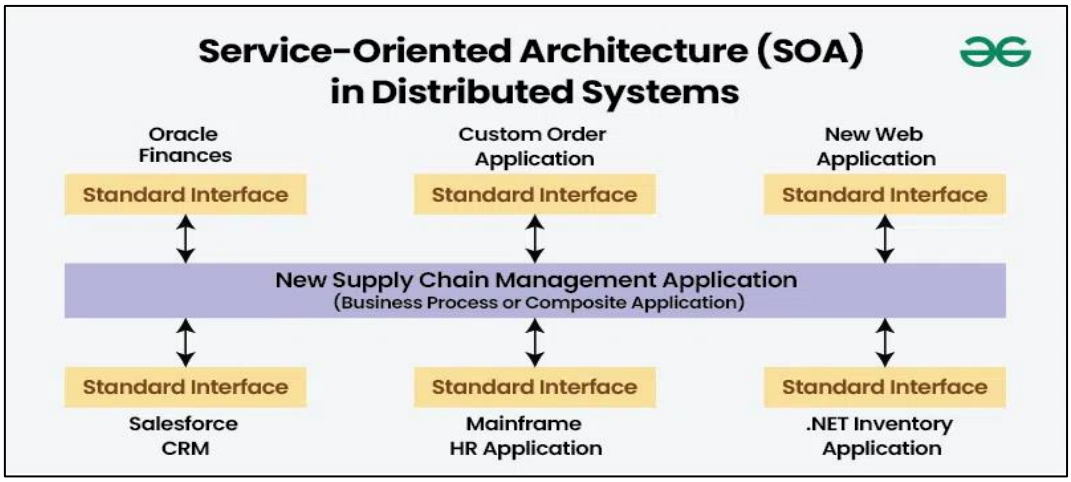
Examples of Data-Centic Architecture in Distributed Systems

1) **Relational Databases**: Systems like MySQL, PostgreSQL, and Oracle use Data-Centric Architecture to manage and store structured data efficiently, providing consistent access and integration across applications.
2) **Data Warehouses**: Platforms such as Amazon Redshift and Google BigQuery are designed to centralize and analyze large volumes of data from various sources, enabling complex queries and data analysis.
3) **Enterprise Resource Planning (ERP) Systems**: ERP systems like SAP and Oracle ERP integrate various business functions (e.g., finance, HR, supply chain) around a centralized data repository to support enterprise-wide operations and decision-making.

2.2.5. Service-Oriented Architecture (SOA) in Distributed Systems

Service-Oriented Architecture (SOA) is a design paradigm in distributed systems where software components, known as "services," are provided and consumed across a network. Each service is a discrete unit that performs a specific business function and communicates with other services through standardized protocols.

- In SOA, the system is structured as a collection of services that are loosely coupled and interact through well-defined interfaces. These services are independent and can be developed, deployed, and managed separately.
- They communicate over a network using standard protocols such as HTTP, SOAP, or REST, allowing for interoperability between different systems and technologies.



Service-Oriented Architecture (SOA)

Key Principles of Service-Oriented Architecture (SOA) in Distributed Systems

1) **Loose Coupling**:
   - **Function**: Services are designed to be independent, minimizing dependencies on one another.
   - **Principle**: Changes to one service do not affect others, enhancing system flexibility and maintainability.
2) **Service Reusability**:
   - **Function**: Services are created to be reused across different applications and contexts.
   - **Principle**: Reduces duplication of functionality and effort, improving efficiency and consistency.
3) **Interoperability**:
   - **Function**: Services interact using standardized communication protocols and data formats, such as XML or JSON.
   - **Principle**: Facilitates communication between diverse systems and platforms, enabling integration across heterogeneous environments.
4) **Discoverability**:
   - **Function**: Services are registered in a service directory or registry where they can be discovered and invoked by other services or applications.
   - **Principle**: Enhances system flexibility by allowing dynamic service discovery and integration.

5) **Abstraction**:
   - **Function**: Services expose only necessary interfaces and hide their internal implementation details.
   - **Principle**: Simplifies interactions between services and reduces complexity for consumers.

Advantages and Disadvantages of Service-Oriented Architecture (SOA) in Distributed Systems

Advantages:

1) **Flexibility**: Loose coupling allows for easier changes and updates to services without impacting the overall system.
2) **Reusability**: Services can be reused across different applications, reducing redundancy and development effort.
3) **Scalability**: Services can be scaled independently, supporting dynamic load balancing and efficient resource utilization.
4) **Interoperability**: Standardized protocols enable integration across various platforms and technologies, fostering collaboration and data exchange.

Disadvantages:

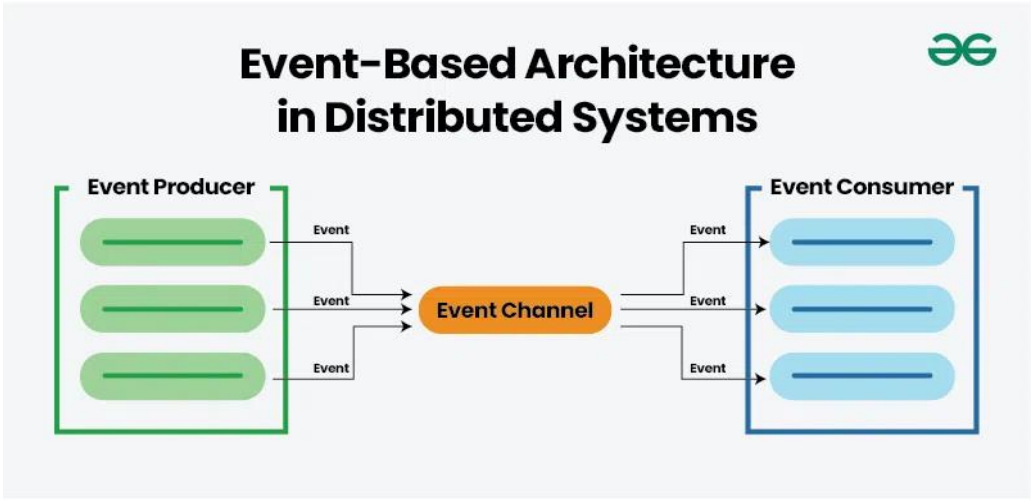
1) **Complexity**: Managing multiple services and their interactions can introduce complexity, requiring effective governance and orchestration.
2) **Performance Overhead**: Communication between services over a network can introduce latency and overhead, affecting overall system performance.
3) **Security**: Ensuring secure communication and consistent security policies across multiple services can be challenging.
4) **Deployment and Maintenance**: Deploying and maintaining a distributed collection of services requires robust infrastructure and management practices.

Examples and Use Cases of Service-Oriented Architecture (SOA) in Distributed Systems

1) **Enterprise Systems**: SOA is commonly used to integrate various enterprise applications such as ERP, CRM, and HR systems, allowing them to work together seamlessly.
2) **Web Services**: Many modern web applications leverage SOA principles to interact with external services via APIs, enabling functionalities such as payment processing, data retrieval, and authentication.

2.2.6. Event-Based Architecture in Distributed Systems

Event-Driven Architecture (EDA) is an architectural pattern where the flow of data and control in a system is driven by events. Components in an EDA system communicate by producing and consuming events, which represent state changes or actions within the system.



Event-Based Architecture

Key Principles of Event-Based Architecture in Distributed Systems

1) **Event Producers**: Components or services that generate events to signal state changes or actions.
2) **Event Consumers**: Components or services that listen for and react to events, processing them as needed.
3) **Event Channels**: Mechanisms for transmitting events between producers and consumers, such as message queues or event streams.
4) **Loose Coupling**: Producers and consumers are decoupled, interacting through events rather than direct calls, allowing for more flexible system interactions.

Advantages and Disadvantages of Event-Based Architecture in Distributed Systems

Advantages:

1) **Scalability**: Supports scalable and responsive systems by decoupling event producers from consumers.
2) **Flexibility**: Allows for dynamic and real-time processing of events, adapting to changing conditions.
3) **Responsiveness**: Enables systems to react immediately to events, improving responsiveness and user experience.

Disadvantages:

1) **Complexity**: Managing event flow, ensuring reliable delivery, and handling event processing can be complex.
2) **Event Ordering**: Ensuring correct processing order of events can be challenging, especially in distributed systems.
3) **Debugging and Testing**: Troubleshooting issues in an event-driven system can be difficult due to asynchronous and distributed nature.

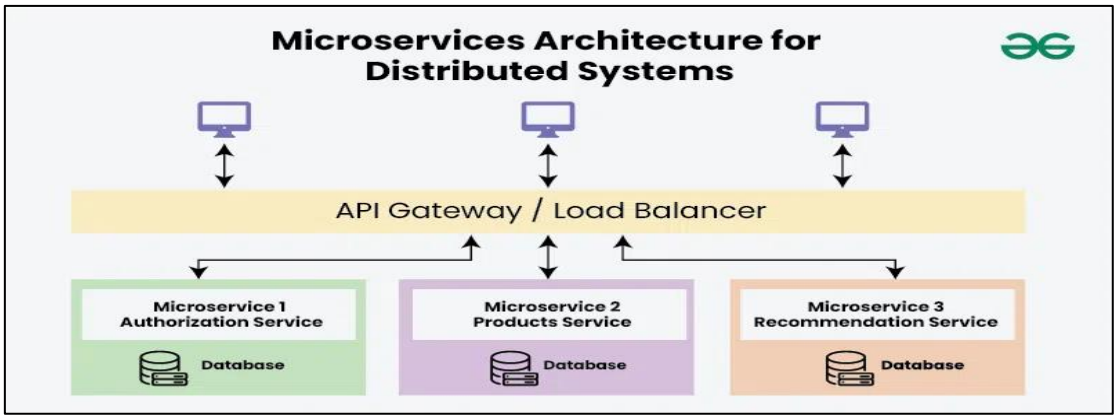
Examples and Use Cases of Event-Based Architecture in Distributed Systems

1) **Real-Time Analytics**: Systems like stock trading platforms use EDA (Event-Driven Architecture) to process and respond to market events in real time.

2) **IoT Systems**: Internet of Things (IoT) applications use EDA to manage and respond to data from various sensors and devices.
3) **Fraud Detection**: Financial institutions use EDA to detect and respond to suspicious activities or anomalies in real time.

2.2.7. Microservices Architecture for Distributed Systems

Microservices Architecture is a design pattern where an application is composed of small, independent services that each perform a specific function. These services are loosely coupled and interact with each other through lightweight communication protocols, often over HTTP or messaging queues.



Microservices Architecture

Key Principles of Microservices Architecture for Distributed Systems

1) **Single Responsibility**: Each microservice focuses on a single business capability or function, enhancing modularity.
2) **Autonomy**: Microservices are independently deployable and scalable, allowing for changes and updates without affecting other services.
3) **Decentralized Data Management**: Each microservice manages its own data, reducing dependencies and promoting scalability.
4) **Inter-service Communication**: Services communicate through well-defined APIs or messaging protocols.

Advantages and Disadvantages of Microservices Architecture for Distributed Systems

Advantages:

a) **Scalability**: Services can be scaled independently based on demand, improving resource utilization.
b) **Resilience**: Failure in one service does not necessarily impact others, enhancing system reliability.
c) **Deployment Flexibility**: Microservices can be developed, deployed, and updated independently, facilitating continuous delivery.

Disadvantages:

a) **Complexity**: Managing multiple services and their interactions can be complex and requires effective orchestration.
b) **Data Consistency**: Ensuring data consistency across services can be challenging due to decentralized data management.

c) **Network Overhead**: Communication between microservices can introduce latency and require efficient handling of network traffic.

Examples of Microservices Architecture for Distributed Systems

a) **E-Commerce Platforms**: Platforms like Amazon use microservices to handle different aspects of their operations, such as user authentication, payment processing, and order management.
b) **Streaming Services**: Companies like Netflix employ microservices to manage different functionalities, such as recommendation engines, content delivery, and user interfaces.
c) **Financial Services**: Banks and financial institutions use microservices to manage various functions, including transaction processing, customer management, and compliance.

2.3. Design principles of distributed systems architecture

Designing effective distributed systems requires careful consideration of various features and design elements to ensure seamless operation, scalability, and fault tolerance. Here are key features and design elements crucial for distributed systems:

1) **Scalability:** Scalability in distributed systems refers to the system's ability to handle an increasing amount of workload or growth in users and data without sacrificing performance. Achieving scalability is crucial for applications and services to meet the demands of a growing user base and handle varying workloads efficiently. There are two common approaches to scalability.
    a) **Horizontal scalability:** Horizontal scalability–or "scaling out"–involves adding more nodes or servers to a distributed system. Common in cloud environments, where new instances can be added to a cluster automatically to handle increased demand.
    b) **Vertical scalability:** Vertical scalability–or "scaling up"–involves increasing the resources (CPU, RAM, etc.) of existing nodes in the system. This is typically achieved either by adding to the CPU or memory capacity of an existing VM or by purchasing a machine with greater computing resources. Vertical scaling generally requires a much lower initial investment of time and financial resources.
2) **Fault tolerance:** Fault tolerance in distributed systems ensures the system's continued operation despite hardware failures, software errors, or network issues. It removes all single points of failure so the system operates normally even when any individual component malfunctions or becomes unavailable. Implementing fault tolerance requires meticulous architecture design and comprehensive planning for worst-case scenarios.
3) **Replication:** Duplicating aspects of a system, such as hardware and software components, network paths, data stores, and power sources, is essential for redundancy. It can also involve replicating entire systems or subsystems to ensure high availability.
4) **Checkpoints:** Checkpointing refers to periodically saving the state of a distributed system or application for recovery in case of a failure. You can use checkpoints to roll back to a consistent state after a failure occurs.
5) **Failover:** Failover provides a secondary component or service that automatically assumes the functions of the primary system in the event of a primary system failure. This typically involves utilizing a “heartbeat” system between at least two servers. While the “pulse” from the primary

server continues, backup servers remain on standby, ready to come online in the event of a disruption in the service.

6) **Heterogeneity:** The Internet enables users to access services and run applications over a heterogeneous collection of computers and networks. For eg., a computer attached to an Ethernet has an implementation of the Internet protocols over the Ethernet, whereas a computer on a different sort of network will need an implementation of the Internet protocols for that network.
7) **Openness:** The openness of a computer system is the characteristic that determines whether the system can be extended and re-implemented in various ways. The openness of distributed systems is determined primarily by the degree to which new resource-sharing services can be added and be made available for use by a variety of client programs.
8) **Security:** Many of the information resources that are made available and maintained in distributed systems have a high essential value to their users. Their security is therefore of considerable importance. Security for information resources has three components: confidentiality, integrity, and availability.
9) **Failure handling:** Failures in a distributed system are partial – that is, some components fail while others continue to function. Therefore the handling of failures is particularly difficult.
10) **Concurrency:** Both services and applications provide resources that can be shared by clients in a distributed system. Object that represents a shared resource in a distributed system must be responsible for ensuring that it operates correctly in a concurrent environment. This applies not only to servers but also to objects in applications. Therefore any programmer who takes an implementation of an object that was not intended for use in a distributed system must do whatever is necessary to make it safe in a concurrent environment.
11) **Transparency:** Transparency can be achieved at two different levels. Easiest to do is to hide the distribution from the users. The concept of transparency can be applied to several aspects f a distributed system.
    a) Location transparency: The users cannot tell where resources are located
    b) Migration transparency: Resources can move at will without changing their names
    c) Replication transparency: The users cannot tell how many copies exist.
    d) Concurrency transparency: Multiple users can share resources automatically.
    e) Parallelism transparency: Activities can happen in parallel without users knowing.
12) . **Quality of service:** Once users are provided with the functionality that they require of a service, such as the file service in a distributed system, we can go on to ask about the quality of the service provided. Adaptability to meet changing system configurations and resource availability has been recognized as a further important aspect of service quality.
13) **Reliability:** One of the original goals of building distributed systems was to make them more reliable than single-processor systems. Data entrusted to the system must not be lost or garbled in any way, and if files are stored redundantly on multiple servers, all the copies must be kept consistent. In general, the more copies that are kept, the better the availability, but the greater the chance that they will be inconsistent, especially if updates are frequent.
14) **Performance:** Always the hidden data in the background is the issue of performance. In particular, when running a particular application on a distributed system, it should not be worse than running the same application on a single processor. Unfortunately, achieving this is easier said than done.